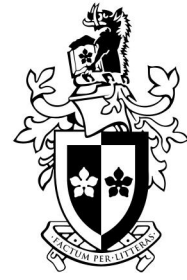


School of Science, Computing and Engineering Technologies
Software Architecture and Application
Assignment 2



Smart Fleet Management (SmartFM) Object-Oriented System Design

Lecturer: Mr.Pham Thai Ky Trung

Course: Software Architecture and Application

CourseID: SWE30003

Dates: Mondays, 13:00 - 16:00 p.m

Team Members: Lu Nhat Hoang - 105234956

Duong Nguyen Dang - 1055084444

Truong Ngoc Gia Hieu - 105565520

Start Date: June 27th, 2026

End Date:

Submission Date:

Word Count: words

Executive Summary

This document presents the comprehensive object-oriented design of the SmartFM Fleet Management System, demonstrating a highly cohesive, scalable, and loosely coupled system architecture. We implement the Model-View-Controller (MVC) structural pattern to cleanly segregate domain logic, command-line user interfaces, and centralized data layers. System instantiation is controlled via the Singleton pattern for physical database gateways, while a dynamic Factory Method manages user role polymorphism. Behavioral flexibility is handled through the Strategy pattern to isolate real-time telemetry alert rules. All architectural decisions adhere to SOLID principles and Arthur Riel's nine core heuristics, verified across four fundamental system sequence scenarios.

TABLE OF CONTENT

Part A: Introduction	5
1.1 Context and Background	5
1.2 Problem Analysis	5
1.2.1 Data Integrity and Object Initialization Constraints	5
1.2.2 Real-Time GPS Tracking and Network Latency Risks	6
1.2.3 Logic Errors in Revenue and Dynamic Pricing Calculations	6
1.3 Purpose and Architectural Scope	6
1.3.1 In-Scope	7
1.3.2 Out-of-Scope	7
1.4 System Architectural Roadmap	7
1.5 Object Design Considerations and Guidelines	7
1.5.1 Architectural Naming Conventions	8
1.5.2 Operational Boundary Cases	8
1.5.2.1 Vehicle and GPS Constraints	8
1.5.2.2 Order and Scheduling Constraints	9
1.5.2.3 Revenue and Pricing Constraints	9
1.5.3 System Exception Handling Framework	10
Part B: Architectural Design Decisions and Justifications ..	11
2.1 Assumptions	11
2.2 Simplifications	12
2.3 Design Justification	12
2.4 Discarded Class List	13
Part C: Candidate Classes	13
3.1 Candidate Class List	14
3.2 CRC Cards	15
3.2.1 User	15
3.2.2 Driver	16
3.2.3 FleetManager	16
3.2.4 SystemAdmin	17
3.2.5 Vehicle	17
3.2.6 VehicleType	18
3.2.7 BranchHub	18

3.2.8 MaintenanceRecord	18
3.2.9 TripAssignment	19
3.2.10 Database	19
3.2.11 Notification	20
Part D: Design Quality	20
4.1 Design Heuristics	20
4.2 Cohension	21
4.3 Coupling	21
4.4 SOLID Principles Analysis	21
Part E: Design Patterns	22
5.1 Sngleton Design Pattern	22
5.2 Factory Method Design Pattern	22
5.3 Behavioral Patterns	22
5.4 Structural Patterns	23
Part F: System Architecture and Verification	23
6.1 Overall UML Diagram	23
6.2 System Dynamic Verification	24
6.2.1 System Boot-strap and Initalization	24
6.2.2 Vehicle and Driver Registration	26
6.2.3 Trip Creation and Allocation	27
6.2.4 Live Telemetry and Automated Alerts	29
Part G: Conclusion	30

Part A: Introduction

1.1 Context and Background

While the first assignment required the development of a comprehensive Software Requirements Specification (SRS) document using the Tasks and Support technique to expand the business operations of ABC-Trans Company, the second report shifts the focus to establishing a high-level object-oriented design for the centralized Smart Fleet Management System (SmartFM). Based on the scenario, the company faces the following issues including operational bottlenecks, customer dissatisfaction, and restricted organizational scalability due to fragmented, manual processes across vehicle allocation, driver tracking, order processing, and branch-specific workflows. Consequently, the SmartFM system is generated to integrate in-branch, web, and mobile applications into a unified platform that optimizes resource scheduling, provides real-time shipment visibility, and aggregates comprehensive utilization statistics.

In detail, the second document emphasizes the initial object-oriented system design for the centralized platform by utilizing the Responsibility-Driven Design (RDD) framework. Furthermore, the proposed design maps project requirements and logistics workflows into a clean taxonomy of structural classes, data-holders, and dynamic controllers based on behavioral obligations and object collaborations over premature data structures. Additionally, foundational design patterns and heuristics are integrated to enforce the Open-Closed Principle, delivering a decoupled blueprint that minimizes technical debt and secures a stable foundation for downstream system implementation.

1.2 Problem Analysis

1.2.1 Data Integrity and Object Initialization Constraints

Next, the generation of core entities including *Vehicle* and *Order* objects presents several severe architectural risks if input validation is deferred. Furthermore, relying deeply on internal business logic layers permits malformed data to penetrate the system core, unnecessarily increasing system complexity and compounding technical debt. Specifically, the corrupt data can easily bypass early layers because of the absence of an immediate validation gate during the object compilation and memory allocation phases.

For instance, if a vehicle object is initialized with invalid physical dimensions or capacities, this can ripple through downstream fleet registry states, causing hard-to-debug failures in dispatching algorithms. Additionally, this potential failure requires a deterministic approach to block improper initialization instantly at the threshold, ensuring the system enforces strict object constraints before any memory resource is wasted.

1.2.2 Real-Time GPS Tracking and Network Latency Risks

Subsequently, managing real-time GPS data from many vehicles at the same time generates various risks with the order of incoming data. While GPS devices on vehicles constantly send location updates to the system under normal operations, unstable mobile networks including bad signal zones, lost data, and slow connections cause updates to arrive out of their original order. Notably, a significant logic error happens where older location data can overwrite the newest location state if the platform simply processes these coordinates based on arrival time. As a result, this data confusion breaks the live tracking display, making vehicles look like they are jumping backward on the map, and ruins travel history statistics. Therefore, the system needs a strict time-check filter to quickly drop out-of-order data and protect the true location history, creating a clear need for *NetworkLatencyException* and *StateRegressionException*.

1.2.3 Logic Errors in Revenue and Dynamic Pricing Calculations

Then, processing financial billing and dynamic pricing reveals major risks if the system accepts invalid numbers. In detail, the core billing engine must follow strict and unbreakable rules. For instance, the total invoice amount must always be greater than zero and peak-hour price multipliers must never go over a limit of 2.5. However, errors or bad inputs can easily slip in while processing many transactions from different APIs at the same time. Furthermore, invalid numbers such as negative prices or illegal surcharges can bypass initial checks and enter the system's core memory when handling many transactions from different APIs at the same time. Consequently, this data pollution permanently corrupts active invoice objects, which are then saved directly to the database. Once these corrupted records are saved, they cause cascading errors in tax calculations and ruin financial reports. To solve this problem, the architecture must place a strict validation shield at the system boundary. Besides, any request that breaks these financial rules must be stopped immediately before it can affect the database, which directly justifies the need for *BusinessBoundaryException*.

1.3 Purpose and Architectural Scope

Next, the primary purpose of the second report focuses on translating functional system requirements from the first assignment to a clear network of software classes and defining their relationships. By leveraging proven software design patterns and heuristics, the desired design aims to strike a perfect balance between supporting complex scheduling rules, enabling real-time tracking, and ensuring high code maintainability.

In terms of architectural scope, the centralized platform is designed strictly to establish a high-level logical system architecture for the centralized Smart Fleet Management System (SmartFM) by utilizing the Responsibility-Driven Design (RDD) framework. Specifically, the boundaries of the design are explicitly defined into two parts: In-Scope and Out-of-Scope.

1.3.1 In-Scope

- Mapping functional logistics workflows and requirements into a clean hierarchy of structural classes, data-holders, and dynamic controllers.
- Defining clear behavioral obligations, system responsibilities, and object collaborations using RDD heuristics.
- Enforcing the Open-Closed Principle through foundational design patterns to deliver a decoupled structural blueprint.

1.3.2 Out-of-Scope

- Designing detailed database structures, concrete tables, or storage schemas.
- Developing user interface (UI/UX) wireframes, layouts, or frontend component animations.
- Writing low-level concrete source code or full field definitions for the programming environment.

1.4 System Architectural Roadmap

Subsequently, the document showcases a logical architectural roadmap to systematically transition the Smart Fleet Management System (SmartFM) from abstract functional requirements into a highly scalable object-oriented software structure. Notably, the platform design initiates with the application of the Responsibility-Driven Design (RDD) framework, using Class-Responsibility-Collaborator (CRC) cards to systematically decompose system workflows into explicit object stereotypes and define their operational boundaries.

In addition, the application is formalized into a comprehensive UML Class Diagram to visualize the network of software classes while enforcing structural integrity through inheritance hierarchies, aggregations, and associations. Finally, the intelligence management system integrates foundational object-oriented design patterns and architectural heuristics to successfully decouple system dependencies, manage complex real-time fleet logistics, and guarantee long-term code maintainability.

1.5 Object Design Considerations and Guidelines

Next, the Object Design Considerations and Guidelines part highlights the foundational object design rules, operational boundaries, and exception-handling frameworks implemented within the intelligent management system. In addition, its main purpose is to enforce strict architectural uniformity and system resilience, preventing technical debt across all modeled software components. Consequently, this part guarantees that the subsequent logical design remains robust, cohesive, and easily maintainable under both standard and exceptional operational workflows by defining explicit naming conventions, data guardrails, and error-propagation mechanisms.

1.5.1 Architectural Naming Conventions

Component Type	Naming Rule	Description and Casing Mechanics	Contextual Application
Classes/Entities	Noun	Names for classes are to be descriptive and simple as possible. Class names are to be capitalised.	- class Vehicle - class DeliveryRoute
Interfaces	Adjective/Noun	Similar to Classes/Entities rules	- interface RoutingService - interface DataAccess
Methods/Functions	Verb + Noun	Keeps the initial letter of the first word in lowercase while capitalizing the first letter of all subsequent words.	- function optimizeRoute() - function assignDriver()
Attributes/Variables	Noun	Variable names are to be short and meaningful and start with an underscore character.	- attribute _currentStatus - attribute _licensePlate

1.5.2 Operational Boundary Cases

In this section, the Smart Fleet Management platform must enforce strict validation guardrails directly onto object attributes during instantiation and state transitions to safeguard internal domain entities from structural corruption. Therefore, the system should embed the following constraints within the domain model's logic instead of treating boundary conditions as abstract external requirements. Specifically, the platform boundary metrics across the core subsystems are structured below:

1.5.2.1 Vehicle and GPS Constraints

This subpart governs the physical requirements and live location data streams to guarantee the digital model accurately mirrors the ABC Trans company fleet assets:

- Vehicle Capacity Boundaries: Prevents registering invalid truck dimensions. For instance, *Vehicle._maxPayloadCapacity* $\leq 30,000\text{ kg}$ and *Vehicle._volumetricLimit* $\leq 120\text{m}^3$ indicate that any instantiation sequence exceeding the above metrics is intercepted, blocking object creation and throwing a capacity exception.

- **Mileage State Integrity:** Guards against mileage corruption or regression. For example, variable named *Vehicle._currentMileage* must be larger than its current state value which showcases that any lower or equal inputs are rejected to halt the transaction and preserve historical ledger integrity.
- **Telemetry Velocity Cap:** Filters out impossible speeds caused by faulty GPS sensors. In detail, the metric *TelemetryData._calculatedVelocity* $\leq 120\text{km/h}$ highlights that any sequential ingestion packet implying a higher traveling speed is immediately discarded at the API edge layer and logs a tracking anomaly.
- **Geofence Spatial Bounding:** Restricts system processing to regional operation boundaries. Particularly, *GPSCoordinate._latitude* and *GPSCoordinate._longitude* residing inside Vietnam's designated spatial polygon indicate that any out-of-bounds tracking coordinates are dropped immediately to isolate erratic streams.

1.5.2.2 Order and Scheduling Constraints

Subsequently, this section enforces the dynamic order validation, cargo allocation metrics, and temporal routing parameters to ensure dispatch operational efficiency:

- **Cargo Physicality Guardrail:** Removes zero or negative items to avoid downstream mathematical errors. Specifically, *CargoItem._weight* > 0 and *CargoItem._volume* > 0 mean that any order compilation violating these thresholds is aborted at the boundary layer.
- **Temporal Allocation Boundary:** Aims to prevent assigning trips into the past or invalid timeline sequences. In depth, the metric *DeliveryRoute._targetDeliveryWindow* $\geq \text{Current system clock}$, demonstrating that any unroutable historical constraints block route creation entirely.
- **Fleet Load Validation:** Checks aggregate cargo distribution against the truck's technical limits. For instance, *Sum of CargoItem._weight* $\leq \text{Assigned Vehicle._maxPayloadCapacity}$ displays that any overloading attempt blocks the dispatch allocation loop while maintaining the vehicle's state as *Available*.
- **Resource Exclusivity Constraint:** Ensures assets and drivers are not double-booked concurrently. For instance, the condition where concurrent delivery window overlap equals 0 showcases that any overlapping assignment is blocked by domain-level validation.

1.5.2.3 Revenue and Pricing Constraints

Subsequently, this part outlines the financial guardrails, dynamic pricing parameters, and invoice generation metrics to guarantee transactional transparency:

- **Tariff Floor Regulation:** Prevents system execution of zero-dollar or negative-value commercial trips. Specifically, metrics *Invoice._totalAmount* > 0 and *PriceQuote._baseRate* > 0 demonstrate that any transaction block dropping below these pricing thresholds is aborted immediately to avoid revenue leakage.
- **Surcharge Scaling Boundary:** Restricts excessive variable cost inflation during peak demands. In depth, the metric *Invoice._peakSurchargeMultiplier* ≤ 2.5 demonstrates that any dynamic algorithmic price surge exceeding this hard-coded limit is capped at the API layer.
- **Discount Percentage Cap:** Limits the maximum marketing markdown permitted per booking sequence. For instance, the condition where *Promotion._discountPercentage* ≤ 50% showcases that any promotional application exceeding this boundary is rejected by system-level validation.
- **Payment Grace Temporal Threshold:** Regulates the maximum allowable aging window for unpaid corporate accounts. Particularly, the difference between Current system clock and *Invoice._issueDate* ≤ 30 days indicates that exceeding this credit term automatically transitions the account state to *Suspended*.

1.5.3 System Exception Handling Framework

Next, the following part named 'System Exception Handling Framework' aims to explain the centralized error trapping mechanisms, logging protocols, and state recovery workflows to maintain platform resilience:

- **Capacity Limit Exception:** Intercepts out-of-bounds dimensional allocations at the initialization layer. Notably, triggering a *CapacityLimitException* error blocks object compilation instantly and routes the structural payload to the system logs, confirming that no corrupt memory allocation affects the whole system and active fleet domain registry.
- **State Regression Exception:** Rejects outdated and out-of-order telemetry updates that conflict with timeline sequences. Moreover, activating the *StateRegressionException* handler allows the system to halt the current transaction and trigger a rollback, demonstrating that the database preserves data history without causing duplication.
- **Business Boundary Exception:** Handles validation failures related to allocation rules or financial metrics. Furthermore, if the system detects a financial constraint violation such as an operational request with a *negative Invoice._totalAmount*, it aborts the execution line immediately, showcasing that invalid pricing inputs are caught before reaching internal processing engines.
- **Network Latency Exception:** Manages connection drops or timeouts within real-time data pipelines. Particularly, packet timeouts exceeding the edge threshold trigger automatic retries, highlighting that unstable network connections are isolated to protect core system threads.

Part B: Architectural Design Decisions and Justifications

The purpose of this section is to establish the physical boundaries and technical reasons behind the architectural design of the SmartFM system. Particularly, this part highlights the core operational assumptions and practical simplifications made to narrow the project's scope to ensure the proposed design is both realistic and implementable. Additionally, the second part not only provides detailed justifications for the key design decisions and documents the discarded class designs but also demonstrates a deliberate effort to minimize technical debt and maintain an efficient system architecture.

2.1 Assumptions

- A1** The system only processes logistics and transportation workflows for ABC-Trans.
- A2** All vehicles in the fleet are equipped with standard, functional GPS tracking hardware.
- A3** Every vehicle and driver has a unique identification number registered in the system.
- A4** GPS telemetry devices actively and continuously transmit location data over cellular networks.
- A5** Network disconnections or communication delays from vehicles are temporary, not permanent hardware failures
- A6** All central servers and vehicle GPS units utilize a synchronized time source (UTC)..
- A7** Timestamps generated by tracking devices are globally accurate and reliable.
- A8** External partners and supermarket platforms interact with SmartFM strictly through authorized APIs.
- A9** All external API payloads and bulk order data comply with established documentation formats under normal conditions.
- A10** The database and persistent storage layers are fully operational and maintain 100% data integrity.
- A11** The system boundary is the primary layer responsible for validating all inbound transactional adjustments
- A12** Once transaction details successfully pass edge validation and are saved to the database, they cannot be modified by unverified sources
- A13** A driver can only be assigned to manage one vehicle and one active delivery trip at any given time.
- A14** Each delivery order contains exactly one pickup location, one destination, and one assigned customer.
- A15** All invoice calculations assume the currency is fixed and does not require real-time foreign exchange conversion.
- A16** Peak-hour dynamic pricing multipliers are only applied within predefined operational hours configured in the system
- A17** An order status can only transition forward (e.g., from Pending to Dispatched) and cannot move backward once completed.
- A18** The system assumes that all registered drivers hold a valid commercial driving license.

A19 Distances used for route calculations are based on standard digital map data and do not account for road altitude changes.

A20 Once a delivery trip is finalized and marked as completed, the associated customer invoice cannot be deleted from the active records.

2.2 Simplifications

Based on the assumptions made regarding the requirements, there were some simplifications made to the project that allowed my team to structure the application more effectively:

- Information for vehicles and drivers will be stored in a unified format in the database and treated as standard entities in the system to simplify tracking calculations.
- Delayed GPS data packets and out-of-order telemetry signals are considered standard network latency issues, allowing the system boundary to filter them out using a single validation shield.
- Real-Time pricing fluctuations and peak-hour rate changes are processed under a unified schedule-based multiplier config, treating all active delivery routes within the same timeframe equally.
- Completed orders and finalized transaction logs are treated as read-only historical records, bypassing the need to implement complex multi-level rollback or state-reversion logic for finished trips.
- All financial values and distance metrics are stored and processed as simple numerical types, ignoring dynamic external factors such as currency conversion fluctuations or route elevation differences.

2.3 Design Justification

The SmartFM system is architected to run efficiently and reliably during ABC-Trans's daily operations. Based on the assumptions and simplifications generated in previous sections, this section outlines the clear reasons behind our architectural choices:

- **Simplifying Database Structure with Unified Entities:** By storing and displaying vehicles and drivers in a unified format, the database of the management system significantly reduces query overhead and optimizes memory usage during real-time tracking calculations, thereby avoiding potential performance bottlenecks.
- **Protecting the Core System at the Boundary:** Our team decided to implement a strict boundary validation layer between the API and Core Domain Services instead of allowing delayed, corrupted, and out-of-order GPS telemetry signals to penetrate deep into the application core by discarding them early using a single and lightweight validation shield. As a result, the suggested approach maintains clean state transitions within our core domain services.
- **Handling Errors Gracefully with Custom Exceptions:** Our team decided to design custom exceptions including ***NetworkLatencyException*** and ***StateRegressionException*** as an alternative to relying on generic and system-level exceptions that obscure the root cause of an issue. As expected, the suggested solution facilitates clear boundary isolation, simplifies debugging logs, and ensures that the system can gracefully

inform users of transient cellular network dropouts without system-wide failures.

- **Ensuring Data Integrity with Read-Only Transactions:** Restricting completed delivery orders and finalized invoices to a read-only state eliminates the need for a complex and multi-level undo-redo transaction mechanism in the database. Consequently, the design above guarantees 100% data integrity for financial records, protects the persistent storage from unauthorized tampering, and keeps the overall architecture lean and easy to maintain.

2.4 Discarded Class List

Based on the first assignment, our team has targeted several classes to be discarded. Moreover, the refinement ensures our system remains lightweight and focused only on essential features. In detail, the following list outlines the discarded classes along with the technical rationale for their removal:

- ***UndoRedoTransactionManager:***
 - Description: A class intended to manage multi-level rollbacks and modifications for completed delivery orders and financial invoices.
 - Reason for Discarding: Since completed delivery orders and finalized invoices are restricted to a read-only state, data changes are not permitted after finalization which introduce redundant overhead and unnecessary complexity to the database transaction logic.
- ***GenericNetworkErrorHandler:***
 - Description: A global and generic class designed to catch all system-level network exceptions and return standardized error codes to the client.
 - Reason for Discarding: Using a generic handler obscures the specific root causes of critical domain issues. Additionally, this class was replaced by explicit custom exceptions including ***NetworkLatencyException*** and ***StateRegressionException*** to achieve clearer boundary isolation and more descriptive debugging logs.
- ***DriverVehiclePolymorphicMapper:***
 - Description: A complex mapping class designed to handle polymorphic relationships and dynamic structural joins between separate driver and vehicle entities in the database.
 - Reason for Discarding: Following the decision to simplify the database structure by storing and representing vehicles and drivers in a unified format, the need for dynamic polymorphic mapping was completely eliminated. As a result, discarding this class significantly reduced query overhead and optimized memory allocation during real-time tracking calculations.

Part C: Candidate Classes

3.1 Candidate Class List

Based on the first assignment report and specifications from the SmartFM system of ABC-Trans, a list of candidate classes has been identified which represent potential entities, boundaries, and controllers within the system's domain model, specifically focusing on the Fleet Asset Management module.

- Vehicle:
 - Description: Exhibits the physical assets managed within the logistics network which stores critical technical specifications, operational constraints, and availability metrics required for automated trip generation.
- Driver:
 - Description: Showcases the personnel responsible for running the vehicles and executing delivery assignments, tracking qualifications, real-time availability, and regulatory compliance data such as active driving hours.
- Branch/Hub:
 - Description: Displays the physical regional facilities or distribution centers where vehicles and drivers are stationed and managed.
- TripAssignment:
 - Description: An associative entity that captures the dynamic allocation of a specific driver to a designated vehicle for an active delivery order.
- VehicleType:
 - Description: A lookup/descriptive class that defines standard classifications of fleet assets, particularly Rigid, Refrigerated, and Container Truck along with generalized capacity rules.
- MaintenanceRecord:
 - Description: Illustrates the historical and scheduled servicing events for each vehicle, guaranteeing that only assets with *Available* status are dispatched.
- User:
 - Description: Represents the base entity for any system actor interacting with the platform.
 - FleetManager:

- ◆ Description: A specialized user role responsible for managing vehicles, drivers, assigning trips, and monitoring fleet performance.
- SystemAdmin:
 - ◆ Description: A specialized user role responsible for configuring system configurations, managing user accounts, and maintaining overall system security.
- Notification:
 - Description: Depicts system alerts and communication logs dispatched to drivers and managers.
 - SMSNotification:
 - ◆ Description: A specific type of notification sent via telecommunication networks for real-time mobile updates
 - EmailNotification:
 - ◆ Description: A specific type of notification sent via email protocols for official statements and trip records.
- Database:
 - Description: Shows the persistent storage subsystem containing physical data tables and transaction logs.

3.2 CRC Cards

3.2.1 User

Class Name: User Super Class: -	
A user represents an individual registered within the SmartFM system. A user is defined by a unique email address and a set of credentials to maintain secure system boundaries.	
Responsibilities	Collaborators
Knows identity details (User ID, name, email, phone, and role)	NA
Knows account status and authentication states	NA
Can validate credentials during the login and logout lifecycle	Database

• Can update basic profile and security settings	Database
--	----------

3.2.2 Driver

Class Name: Driver Super Class: User	
A driver denotes an employee assigned to operate vehicles and execute delivery logistics within the fleet management system.	
Responsibilities	Collaborators
• Knows license details, availability status, and assigned vehicle ID	NA
• Knows current trip records and history logs	TripAssignment
• Can update operational status such as <i>active</i>, <i>on trip</i>, and <i>offline</i>	Database
• Can accept or update the progress of an assigned trip	TripAssignment, Database

3.2.3 FleetManager

Class Name: FleetManager Super Class: User	
A fleet manager models an administrative user responsible for overseeing physical vehicles, monitoring maintenance status, and organizing delivery trip logistics within the system.	
Responsibilities	Collaborators
• Knows comprehensive fleet statistics, vehicle operational states, and active driver schedules	NA
• Can assign a driver and a vehicle to create a delivery route validation	TripAssignment, Database
• Can trigger or schedule a vehicle servicing request upon resource failure	Vehicle, MaintenanceRecord, Database
• Can track and review real-time asset configurations and system logs	Vehicle, TripAssignment

3.2.4 SystemAdmin

Class Name: SystemAdmin Super Class: User	
A system admin signifies a privileged user responsible for maintaining system integrity, managing user accounts, and configuring global application settings.	
Responsibilities	Collaborators
Knows global configuration settings and access control privilege matrices	NA
Can create, update, or revoke user accounts and system roles	Database
Can back up, restore, or clear system logs and persistent storage data	Database
Can monitor overall infrastructure health and audit trails	Database

3.2.5 Vehicle

Class Name: Vehicle Super Class: -	
A vehicle represents a single mobile asset within the fleet logistics network, tracked by its operational details, maintenance status, and specific type categorization.	
Responsibilities	Collaborators
Knows identification details include license plate, VIN, model, make, and manufacture year	NA
Knows its associated classification parameters	VehicleType
Knows current physical location assignment	Branch/Hub
Knows overall operational status, for instance: <i>available</i>, <i>on trip</i>, and <i>under maintenance</i>	NA
Can verify history log updates for mechanical servicing	MaintenanceRecord, Database

3.2.6 VehicleType

Class Name: VehicleType Super Class: -	
A vehicle type denotes a classification category that defines the structural capacity, regulatory constraints, and load profiles for a subset of fleet vehicles.	
Responsibilities	Collaborators
Knows category specifications (Type name, description, fuel type, and weight limit)	NA
Knows required driver license class configurations for operation	NA
Can calculate optimal load capacities based on dimensions	NA

3.2.7 BranchHub

Class Name: BranchHub Super Class: -	
A branch or hub models a major physical node within the logistics network that functions as an operational center for managing localized vehicles, cargo, and team schedules.	
Responsibilities	Collaborators
Knows branch details (Hub ID, name, physical address, and contact number)	NA
Knows current inventory levels of stationed vehicles	Vehicle
Can assign local parking zones or docks to incoming assets	Vehicle, Database

3.2.8 MaintenanceRecord

Class Name: MaintenanceRecord Super Class: -

A maintenance record signifies a documented log of servicing activities, mechanical repairs, and safety inspections performed on a specific fleet vehicle.	
Responsibilities	Collaborators
Knows record details (Record ID, maintenance type, cost, description, and completion date)	NA
Knows the vehicle associated with this servicing history	Vehicle
Can verify if a scheduled safety inspection is due or overdue	Database

3.2.9 TripAssignment

Class Name: TripAssignment Super Class: -	
A trip assignment models the operational contract that links a specific driver and vehicle to an approved delivery route and tracks its real-time execution lifecycle.	
Responsibilities	Collaborators
Knows assignment details (Trip ID, route details, scheduled start/end times, and priority)	NA
Knows its current lifecycle state including <i>pending</i>, <i>assigned</i>, <i>in transit</i>, <i>completed</i>, and <i>cancelled</i>	NA
Can calculate estimated arrival deviations based on route telemetry	Vehicle, Database
Can update or close out the assignment status upon delivery fulfillment	Driver, Database

3.2.10 Database

Class Name: Database
Super Class: -

A database utility interface manages persistence operations, connection pooling, and transactional query execution across the fleet logistics information system.	
Responsibilities	Collaborators
Knows connection configurations and active session states	NA
Can execute secure transactional queries for system state changes	NA
Can persist or retrieve records for domain objects	Vehicle, Driver, TripAssignment

3.2.11 Notification

Class Name: Notification Super Class: -	
A notification utility models the system service responsible for formatting, dispatching, and tracking alert messages sent to users regarding critical fleet events.	
Responsibilities	Collaborators
Knows notification details such as <i>Notification ID</i>, <i>recipient ID</i>, <i>message payload</i>, and <i>timestamp</i>	NA
Knows the delivery channel configurations including <i>SMS</i>, <i>Email</i>, and <i>In-App Push</i>	NA
Can dispatch real-time alerts upon system state triggers	Database

Part D: Design Quality

4.1 Design Heuristics

H1 Classes like Vehicle, Driver, and Trip capture exactly one distinct real-world fleet management concept.

H2 Vehicle coordinates, driver details, and trip statuses are kept private and accessed only through public interface methods.

H3 Attributes of base classes like `VehicleType` are declared private, preventing direct access from subclasses or external classes.

H4 Naming conventions for actions such as `verifyActiveStatus`, `verifyAvailability`, and `setStatus` are identical across UML diagrams and classes.

H5 Validation errors explicitly state the issue and action, for instance, "*Vehicle is currently on another trip. Please select an idle vehicle.*"

H6 Core workflow lifecycle hooks in base processor classes are strictly abstract or final to prevent derived processors from overriding them with empty/null operations.

H7 Logic is decentralized; coordinators (*sa*, *co*) only orchestrate workflows while domain entities validate their own data like `Vehicle.verifyAvailability`.

H8 General parent classes such as `User` or `VehicleType` are declared abstract to prevent direct instantiation.

H9 Abstract classes in SmartFM are strictly used as inheritance foundations and are never instantiated on their own.

4.2 Cohesion

- **TelemetryProcessor**: Dedicated strictly to receiving and parsing raw IoT telemetry data.
- **Vehicle** and **Driver**: Manage only their respective operational states, attributes, and validations.
- **AlertSystem**: Dedicated solely to processing alert rules independent of data persistence.

4.3 Coupling

- **Controller Pattern**: Centralized coordinators (*sa:SystemAdmin*, *co:Coordinator*) manage flows, preventing direct dependencies between domain entities for example *User* and *VehicleType*.
- **Database Abstraction**: All entities interact with persistence via the generic *db.execute(query)* interface, decoupling business logic from the concrete database engine.

4.4 SOLID Principles Analysis

- **SRP**: Each class has one reason to change like `AlertSystem` only changes if alert rules are updated.
- **OCP**: Open for extension, closed for modification. For example, new `ElectricVehicleType` extends `VehicleType` without changing core registration flows.

- **LSP:** Subclasses seamlessly replace base classes. In detail, *ElectricVehicleType* behaves consistently in registration workflows.
- **ISP:** Clients only depend on what they use, for example, IoT devices stream data through a minimal telemetry interface.
- **DIP:** High-level modules depend on abstractions like controllers call a generic *Database* interface instead of concrete driver APIs.

Part E: Design Patterns

5.1 Singleton Design Pattern

- **Context and Problem:** The SmartFM platform relies on a single relational database instance (*db:Database*) to execute real-world transactional SQL operations. Therefore, instantiating multiple database connection handlers across different controllers risks resource exhaustion, connection leakage, and transactional conflicts.
- **Application and Justification:** The *Database* class is implemented as a Singleton. Its constructor is declared private, and a static global access method (*Database.getInstance()* or *db*) is provided. As a result, this guarantees that a single, unified database gateway is shared across the entire system runtime lifecycle

5.2 Factory Method Design Pattern

- **Context and Problem:** The platform must register and handle distinct user roles include *SystemAdmin*, *Coordinator*, and *Driver*. Additionally, direct, hardcoded instantiation of concrete user classes inside coordinating controllers creates tight coupling and violates the Open/Closed Principle (OCP) when new roles like *MaintenanceStaff* are introduced.
- **Application and Justification:** A helper factory method instantiates the appropriate *User* subclass dynamically based on input roles. Consequently, the innovation decouples coordinating workflows from concrete instantiation rules, simplifying future platform extensions without modifications to core system controller scripts.

5.3 Behavioral Patterns

- **Context & Problem:** The alert engine (*AlertSystem*) needs to dynamically evaluate incoming IoT data streams against highly varied and evolving rules including speed violations, geofence breaches and low battery levels without hardcoding logical checks.
- **Application & Justification:** Rule-checking behaviors are encapsulated behind an *AlertStrategy* interface. In addition, concrete strategy subclasses including *OverspeedStrategy*, *GeofenceStrategy*, and *BatteryStrategy* implement this interface. Furthermore, the *AlertSystem* delegates rule evaluation via polymorphism at runtime, enabling engineers to append or modify alert behaviors without changing the central processing pipeline.

5.4 Structural Patterns

- **Context and Problem:** For ensuring system scalability, the physical user interaction boundaries (View), core system workflows and coordination (Controller), and persistent state schemas (Model) must remain isolated from each other.
- **Application and Justification:** The system cleanly segments responsibilities across three boundaries:
 - ◆ **Models:** Domain state holders (*Vehicle*, *Driver*, *Trip*, *TelemetryRecord*) containing local data and validation rules.
 - ◆ **Views:** Command-line menu classes (*Menu*) responsible for user input capture and feedback rendering.
 - ◆ **Controllers:** Coordination entities (*SystemAdmin*, *TelemetryProcessor*, *Coordinator*) managing workflows and database updates.

Part F: System Architecture and Verification

6.1 Overall UML Diagram

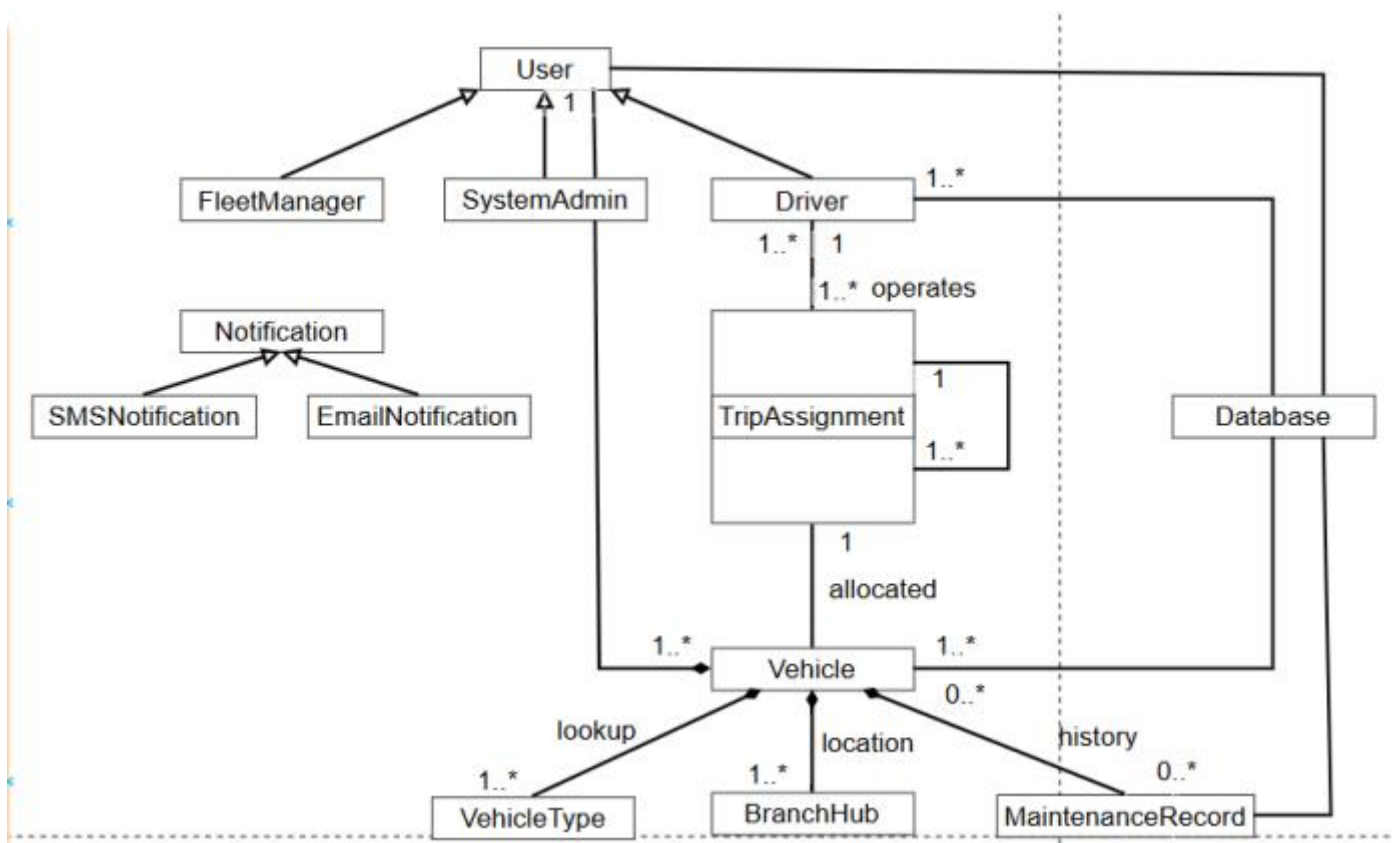


Figure 3: Overall Class Diagram of The System

First and foremost, *Figure 1* above emphasizes the complete architecture of the SmartFM system of the ABC-Trans company, which showcases a highly reliable foundation where the object design structures the platform into distinct operational layers based on the separation of concerns. In detail, the core business domain is driven by the following fundamental entities under the User class: **FleetManager**, **SystemAdmin**, and **Driver**. Furthermore, **Vehicle** and **TripAssignment** also capture the essential fleet logistics and scheduling behaviors.

Regarding the supporting of operational assets, the **VehicleType** class holds fleet classification metadata while the **BranchHub** class defines spatial location mapping for vehicles. Additionally, the **MaintenanceRecord** class explicitly isolates and stores historical vehicle diagnostics and service logs without interfering with core business workflows. Next, in terms of utility and communication infrastructure, the **Database** utility wrapper encapsulates data persistence and centralizes low-level storage interactions. Meanwhile, the **Notification** base class governs system alerting logic, decoupling external communication into concrete **SMSNotification** and **EmailNotification** subclasses to handle real-time messaging dynamically.

6.2 System Dynamic Verification

6.2.1 System Boot-strap and Initalization

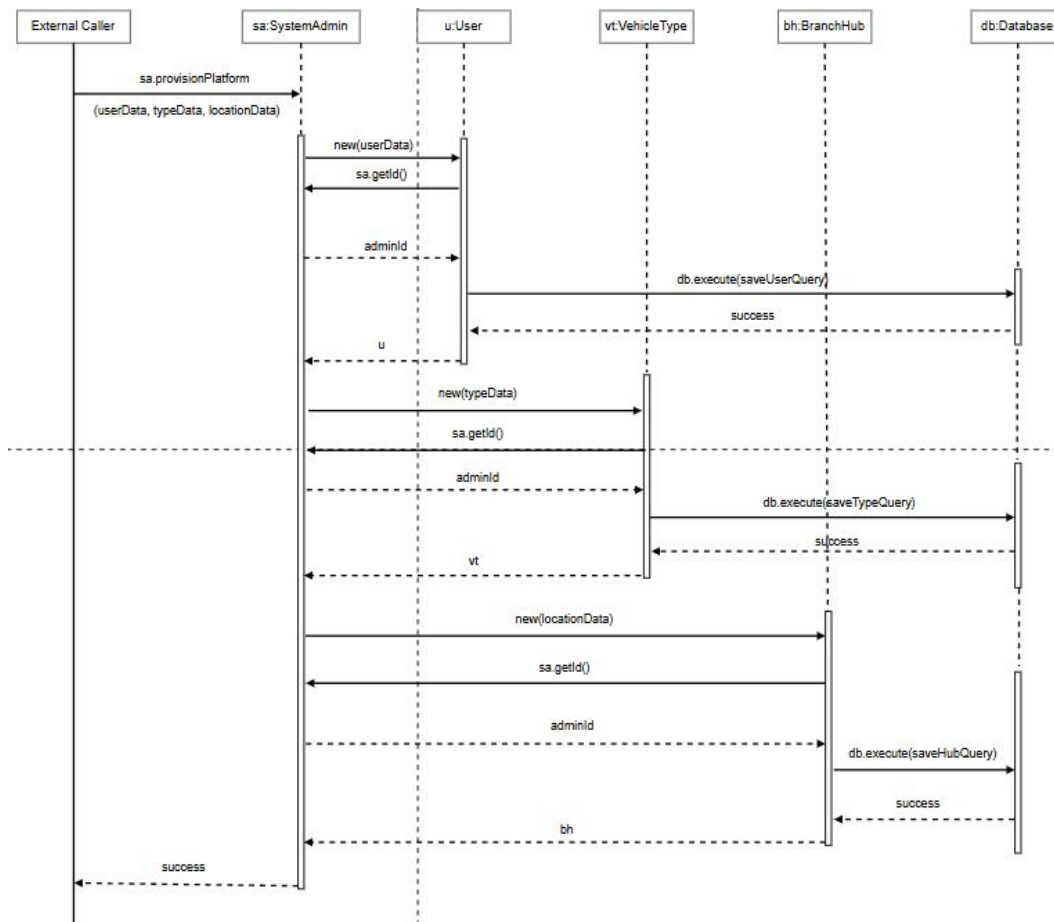


Figure 4: Sequence Diagram for System Bootstrap and Platform Provisioning

The system bootstrap process is initiated when the External Caller invokes `sa.provisionPlatform(userData, typeData, locationData)` to activate the runtime stack of the `sa:SystemAdmin` controller. Upon activation, the controller coordinates the provisioning of the initial administrative user account by calling `new(userData)` on the `u:User` class. In order to establish proper administrative context, the newly spawned `u:User` performs a reverse context check by calling `sa.getId()`, which returns the authenticating `adminId` back to the user object. The user entity then directly persists its state to the database by executing `db.execute(saveUserQuery)`, and upon receiving a success confirmation handshake, returns its completed object reference `u` back to the coordinating administrator controller.

Following user account creation, the coordinator systemically configures the platform's baseline vehicle classifications and geographic branch hubs. First, the controller instantiates `vt:VehicleType` with `new(typeData)`, which calls back to `sa.getId()` for admin verification before invoking `db.execute(saveTypeQuery)` to commit the configuration to the database and return the verified `vt` reference. Next, the controller instantiates the geographical operational region by calling `new(locationData)` on `bh:BranchHub`, which checks back with `sa.getId()` and successfully persists its spatial data using `db.execute(saveHubQuery)`. Once all baseline entities—the administrator user, vehicle categories, and operational branch hubs—are successfully instantiated, verified, and saved, the `sa:SystemAdmin` controller returns a final success confirmation to the External Caller, certifying a completed system boot.

6.2.2 Vehicle and Driver Registration

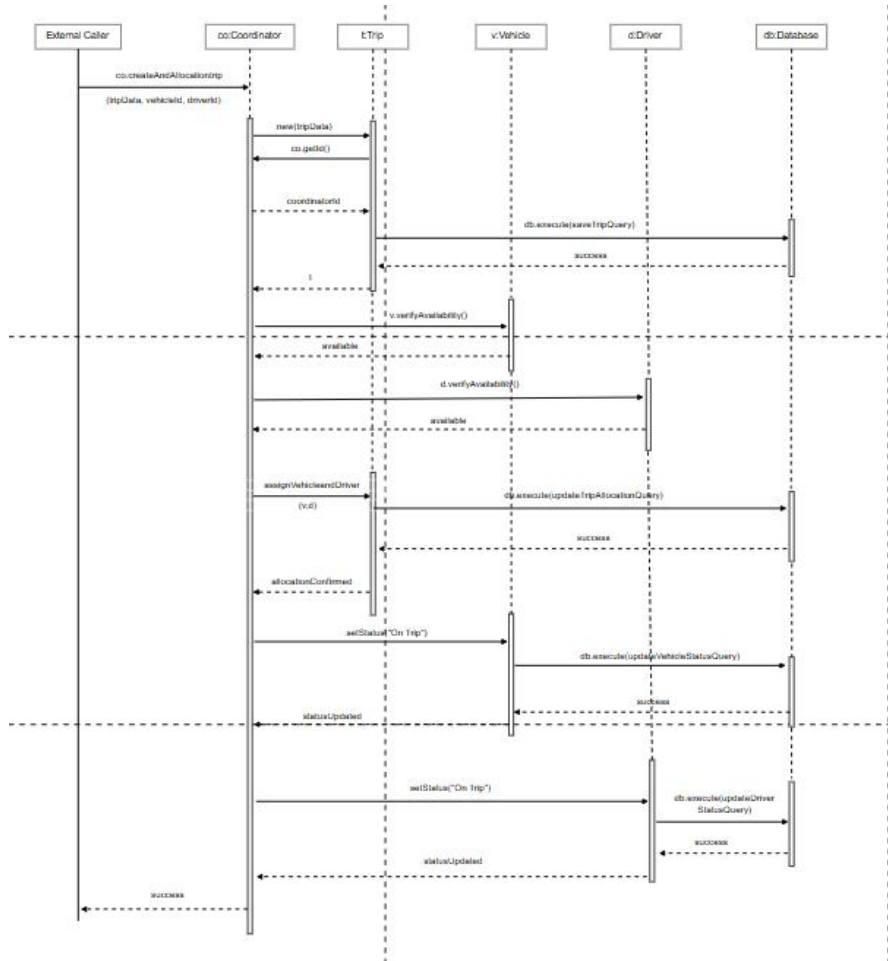


Figure 5: UML Diagram of Vehicle and Driver Registration

The system bootstrap process is initiated when the External Caller invokes `sa.provisionPlatform(userData, typeData, locationData)` to activate the runtime stack of the `sa:SystemAdmin` controller. Upon activation, the controller coordinates the provisioning of the initial administrative user account by calling `new(userData)` on the `u:User` class. To establish proper administrative context, the newly spawned `u:User` performs a reverse context check by calling `sa.getId()`, which returns the authenticating `adminId` back to the user object. The user entity then directly persists its state to the database by executing `db.execute(saveUserQuery)`, and upon receiving a success confirmation handshake, returns its completed object reference `u` back to the coordinating administrator controller.

Following user account creation, the coordinator systemically configures the platform's baseline vehicle classifications and geographic branch hubs. First, the controller instantiates `vt:VehicleType` with `new(typeData)`, which calls back to `sa.getId()` for admin verification before invoking `db.execute(saveTypeQuery)` to commit the configuration to the database and return the verified `vt` reference. Next, the controller instantiates the geographical operational region by calling `new(locationData)` on `bh:BranchHub`, which checks back with `sa.getId()` and successfully persists its spatial data using `db.execute(saveHubQuery)`. Once all baseline entities—

the administrator user, vehicle categories, and operational branch hubs—are successfully instantiated, verified, and saved, the sa:SystemAdmin controller returns a final success confirmation to the External Caller, certifying a completed system boot.

6.2.3 Trip Creation and Allocation

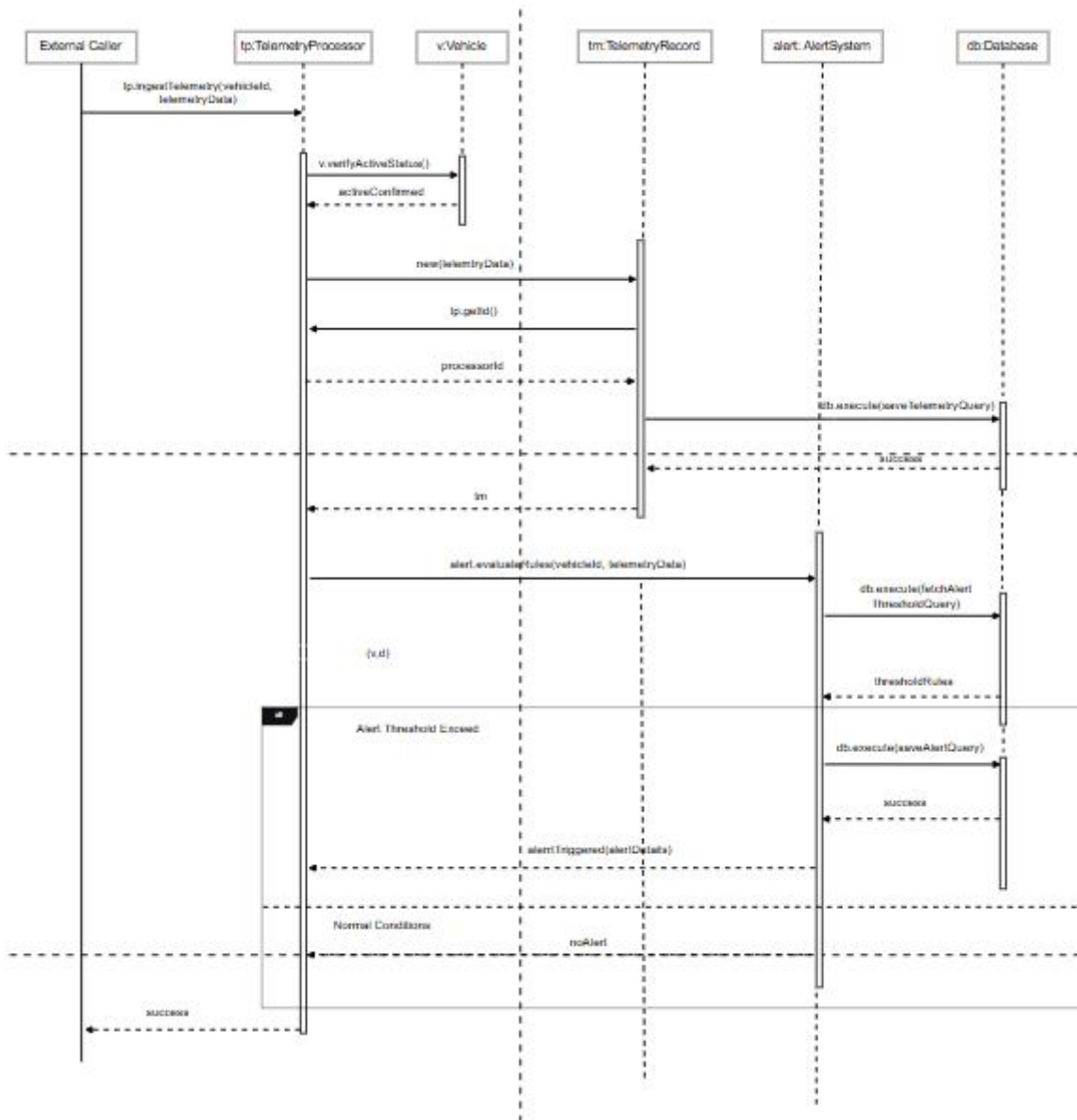


Figure 6: UML Diagram of Trip Creation and Allocation

The workflow depicted in the sequence diagram of **Figure 6** begins when an External Caller invokes the `co.createAndAllocateTrip(tripData, vehicleId, driverId)` method on the central `co:Coordinator` system controller. Upon activation, the coordinator initiates the creation of a new trip instance by calling `new(tripData)` on the `t:Trip` class. With the aim to establish proper administrative context, the newly created `t:Trip` object performs a reverse context lookup by querying `co.getId()`, which returns the authenticated coordinatorId back to the trip entity. This entity then directly persists its

structural information to the relational database using `db.execute(saveTripQuery)`. Once the database issues a success handshake, the fully initialized trip reference `t` is passed back to the coordinator. Immediately following this instantiation, the coordinator systemically verifies the readiness of physical resources by querying vehicle availability through `v.verifyAvailability()` and driver availability through `d.verifyAvailability()`, both of which return an operational status of available.

With resource availability confirmed, the coordinator proceeds to associate the entities by calling `assignVehicleAndDriver(v, d)` directly on the active `t:Trip` instance. The trip object commits this relation by invoking `db.execute(updateTripAllocationQuery)` on the database layer and, upon receiving a success transaction receipt, returns an `allocationConfirmed` response to the controller. The transaction is completed and double-booking is prevented if the coordinator updates the operational state of both allocated assets: it first invokes `setStatus("On Trip")` on the `v:Vehicle` object, which synchronizes its state via `db.execute(updateVehicleStatusQuery)`, and then mirrors this execution sequence by invoking `setStatus("On Trip")` on the `d:Driver` object to trigger `db.execute(updateDriverStatusQuery)`. After both entities acknowledge their updated operational states with a `statusUpdated` reply, the coordination workflow concludes by returning a final success handshake to the External Caller, certifying the successful creation and allocation of the trip.

6.2.4 Live Telemetry and Automated Alerts

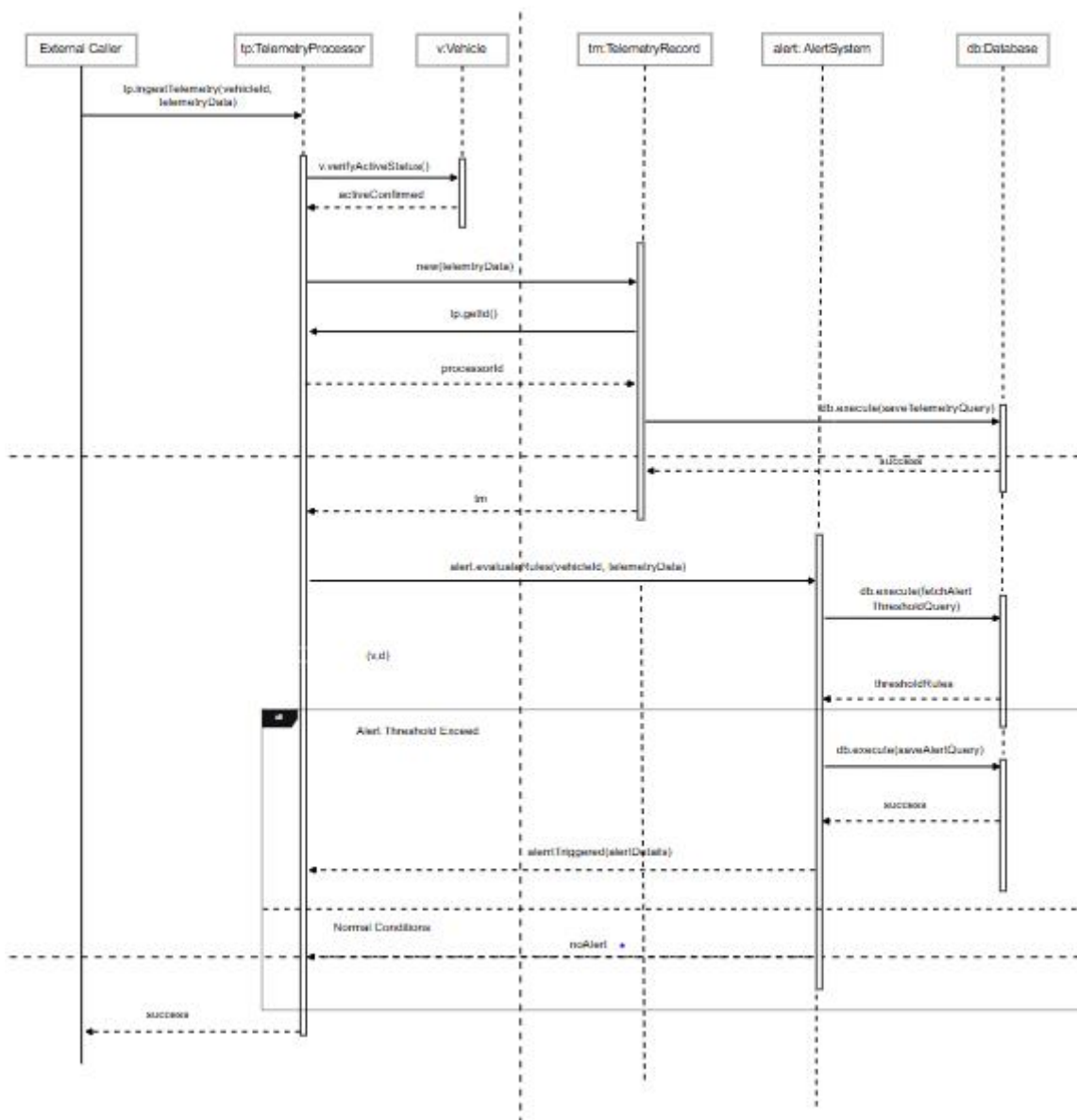


Figure 7: UML Diagram class of Live Telemetry and Automated Alerts

The telemetry ingestion workflow depicted in *Figure 7* is initiated when an External Caller streams raw physical data by invoking `tp.ingestTelemetry(vehicleId, telemetryData)` on the central `tp:TelemetryProcessor` controller. Making ensure data integrity, the processor immediately verifies the target assets by invoking `v.verifyActiveStatus()` on the respective `v:Vehicle` instance. Upon receiving an `activeConfirmed` response, the processor initiates logging by calling `new(telemetryData)` to instantiate a new `tm:TelemetryRecord` object. In binding proper architectural context, the newly spawned `tm` object calls back to `tp.getId()`, which returns the active `processorId`. The record then directly persists its payload to the relational storage layer via `db.execute(saveTelemetryQuery)`. Once the database returns a success handshake, the fully initialized record reference `tm` is handed back to the processor controller.

Immediately after the persistence of raw coordinates, the processor evaluates the incoming metrics against system safety parameters by calling `alert.evaluateRules(vehicleId, telemetryData)` on the `alert:AlertSystem` component. The alert engine queries baseline thresholds from the storage layer using `db.execute(fetchAlertThresholdQuery)` to fetch current `thresholdRules`. The system then processes these rules using a conditional activation block: if an Alert Threshold Exceeded event is identified (e.g., geofence violations or speeding), the engine writes the violation entry using `db.execute(saveAlertQuery)` and returns an active `alertTriggered(alertDetails)` signal back to the controller. Conversely, under Normal Conditions, the engine returns a `noAlert` status. The telemetry cycle completes when the processor wraps up execution and returns a final success response to the External Caller.

Part G: Conclusion

The completed design blueprint for the SmartFM system successfully bridges business requirements with high-quality software architecture. Strict adherence to object-oriented heuristics ensures that the resulting codebase minimizes class dependencies, keeps data tightly encapsulated, and prevents the creation of anti-patterns such as God Classes. Detailed sequence verifications confirm that the execution flows—ranging from initial platform boot to real-time telemetry ingestion—are structurally sound, robust, and mathematically consistent. Engineers can confidently utilize these logical models as a direct, reliable foundation for the concrete implementation and testing phases of development.

